4 Welcome to Docker

This chapter covers

- Understanding the importance of containers
- Recognizing how containers fit into a microservices architecture
- Understanding the differences between a VM and a container
- Using Docker and its main components
- Integrating Docker with microservices

To continue successfully building our microservices, we need to address the portability issue: how are we going to execute our microservices in different technology locations? *Portability* is the ability to use or move software to different environments.

In recent years, the concept of containers has gained more and more popularity, going from a "nice-to-have" to a "must-have" in software architecture. The use of containers is an agile and useful way to migrate and execute any software development from one platform to another (for example, from the developer's machine to a physical or virtual enterprise server). We can replace the traditional models of web servers with smaller and much more adaptable virtualized software containers that offer advantages such as speed, portability, and scalability to our microservices.

This chapter provides a brief introduction to containers using Docker, a technology we selected because we can use it with all the major cloud providers. We will explain what Docker is, how to use its core components, and how to integrate Docker with our microservices. By the end of the chapter, you will be able to run Docker, create your own images with Maven, and execute your microservices within a container. Also, you'll notice you no longer have to worry about installing all the prerequisites your microservices need to run; the only requirement is that you have an environment with Docker installed.

NOTE In this chapter, we will only explain what we are going to use throughout the book. If you are interested in knowing more about Docker, we highly recommend you check out the excellent book, *Docker in Action*, 2nd ed., by Jeff Nickoloff, Stephen Kuenzli, and Bret Fisher (Manning, 2019). The authors provide an exhaustive overview of what Docker is and how it works.

4.1 Containers or virtual machines?

In many companies, virtual machines (VMs) are still the de facto standard for software deployment. In this section, we'll look at the main differences between VMs and containers.

A VM is a software environment that allows us to emulate the operation of a computer within another computer. These are based on a hypervisor that emulates the complete physical machine, allocating the desired amount of system memory, processor cores, disk storage, and other technological resources such as networks, PCI add-ons, and so forth. On the other hand, a container is a package that contains a virtual operating system (OS) that allows us to run an application with its dependent elements in an isolated and independent environment.

Both technologies have similarities, like the existence of a hypervisor or container engine that allows the execution of both technologies, but the way they implement them makes VMs and containers very different. In figure 4.1, you can see the main differences between VMs and containers.

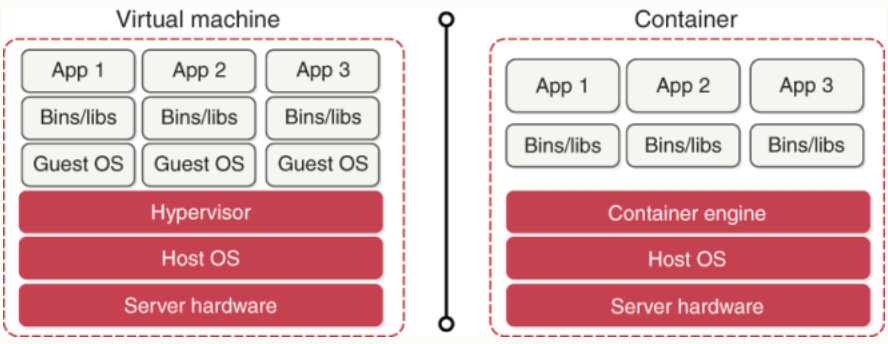


Figure 4.1 Main differences between VMs and containers. Containers don't need the guest OS or the hypervisor to assign resources; instead, they use a container engine.

If we analyze figure 4.1, at first glance, we can see that there isn't that much difference. After all, only the guest OS layer has disappeared in the container, and the hypervisor is replaced by the container's engine. However, the differences between VMs and containers are still enormous.

In a VM, we must set how many physical resources we'll need in advance; for example, how many virtual processors or how many GBs of RAM and disk space we are going to use. Defining those values can be a tricky task, and we must be careful to take into consideration the following:

- Processors can be shared between different VMs.
- Disk space in the VM can be set to use only what it needs. You can define a disk max size, but it will only use the actively consumed space on your machine.
- Reserved memory is total and can't be shared between VMs.

With containers, we can also set the memory and the CPU that we'll need by using Kubernetes, but this isn't required. In case you don't specify these values, the container engine will assign the necessary resources for the container to function correctly. Because containers don't need a complete OS, but can reuse the underlying one, this reduces the load the physical machine must support, as well as the storage space used and the time needed to launch the application. Therefore, containers are much lighter than VMs.

In the end, both technologies have their benefits and drawbacks, and the ultimate decision depends on your specific needs. For example, if you want to handle a variety of operating systems, manage multiple applications on a single server, and execute an application that requires the functionalities of an OS, VMs are a better solution.

In this book, we've chosen to use containers due to the cloud architecture we are building. Instead of virtualizing the hardware as with the VM approach, we will use containers to only virtualize the OS level, which creates a much lighter and faster alternative than what we would be able to run on premises and on every major cloud provider.

Nowadays, performance and portability are critical concepts for decision making in a company. It's important, therefore, to know the benefits of the technologies we are going to use. In this case, with the use of containers with our microservices, we will have these benefits:

- Containers can run everywhere, which facilitates development and implementation and increases portability.
- Containers provide the ability to create predictable environments that are entirely isolated from other applications.
- Containers can be started and stopped faster than VMs, which makes them cloud-native feasible.
- Containers are scalable and can be actively scheduled and managed to optimize resource utilization, increasing the performance and maintainability of the application running within.
- We can realize a maximum number of applications on a minimum number of servers.

Now that we understand the difference between VMs and containers, let's take a closer look at Docker.

4.2 What is Docker?

Docker is a popular open source container engine based on Linux, created by Solomon Hykes, founder and CEO of dotCloud in March, 2013. Docker started as a nice-to-have technology that was responsible for launching and managing containers within our applications. This technology allowed us to share the resources of a physical machine with different containers instead of exposing different hardware resources like VMs.

The support that big companies like IBM, Microsoft, and Google gave to Docker allowed for the conversion of a new technology into a fundamental tool for software developers. Nowadays, Docker continues to grow and currently represents one of the most widely used tools to deploy software with containers on any server.

DEFINITION A *container* represents a logical packaging mechanism, providing applications with everything they need to run.

To understand better how Docker works, it's essential to note that the Docker Engine is the core piece of the entire Docker system. What is the Docker Engine? It is an application that follows the client-server pattern architecture. This application is installed in the host machine and contains the following three critical components: server, REST API, and command-line interface (CLI). Figure 4.2 illustrates these Docker components, as well as others.

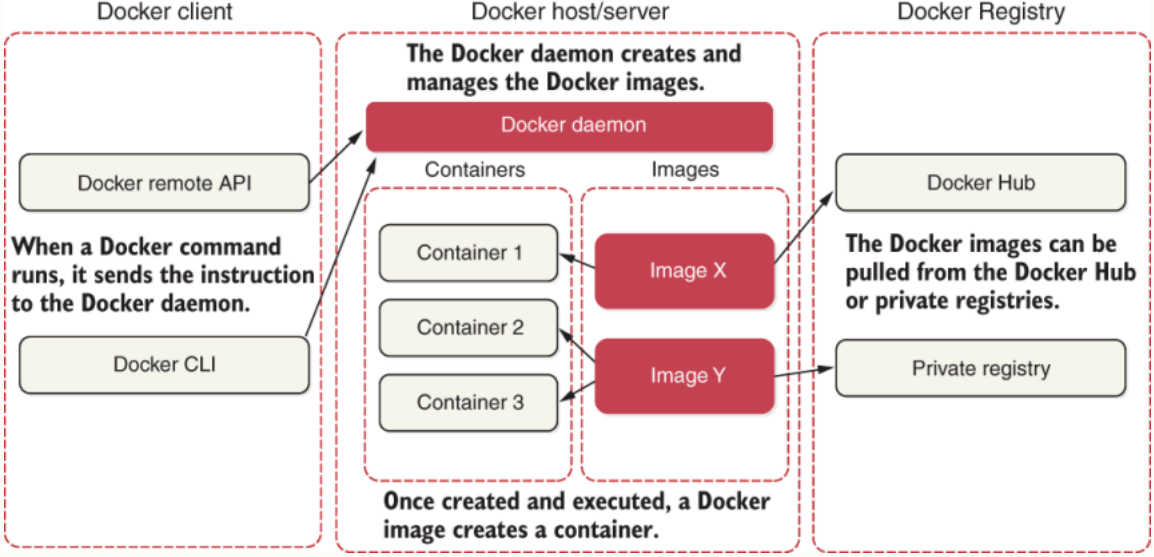


Figure 4.2 Docker architecture composed of the Docker client, the Docker host, and the Docker Registry.

The Docker Engine contains the following components:

- *Docker daemon*—A server, called dockerd, that allows us to create and manage the Docker images. The REST API sends instructions to the daemon, and the CLI client enters the Docker commands.
- *Docker client*—Docker users interact with Docker via a client. When a Docker command runs, the client is in charge of sending the instruction to the daemon.
- *Docker Registry*—The location where Docker images are stored. These registries can be either public or private. The Docker Hub is the default place for the public registries, but you can also create your own private registry.
- *Docker images*—These are read-only templates with several instructions to create a Docker container. The images can be pulled from the Docker Hub, and you can use them as is, or you can modify them by adding additional instructions. Also, you can create new images by using a Dockerfile. Later on, in this chapter, we will explain how to use Dockerfiles.
- *Docker containers*—Once created and executed with the `docker run` command, a Docker image creates a container. The application and its environment run inside this container. In order to start, stop, and delete a Docker container, you can use the Docker API or the CLI.
- *Docker volumes*—Docker volumes are the preferred mechanism to store data generated by Docker and used by the Docker containers. They can be managed using the Docker API or the Docker CLI.
- *Docker networks*—The Docker networks allow us to attach the containers to as many networks as we like. We can see the networks as a means to communicate with isolated containers. Docker contains the following five network driver types: `bridge`, `host`, `overlay`, `none`, and `macvlan`.

Figure 4.3 shows a diagram of how Docker works. Note that the Docker daemon is responsible for all the container's actions. As shown in figure 4.3, we can see that the daemon receives the commands from the Docker client; these commands can be sent via the CLI or the REST APIs. In the diagram, we can see how the Docker images found in the registries create the containers.

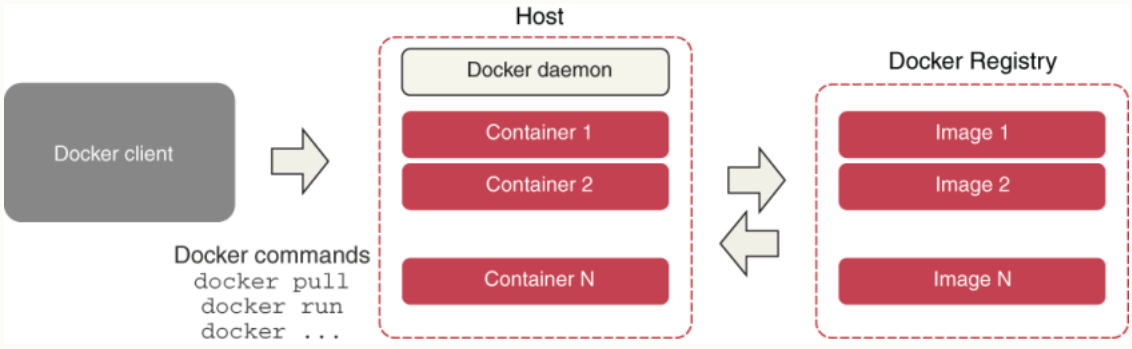


Figure 4.3 The Docker client sends Docker commands to the Docker daemon, and the Docker daemon creates the containers based on the Docker images.

NOTE In this book, we won't teach you how to install Docker. If you don't have Docker already installed on your computer, we highly recommend you look at the following documentation, which contains all the steps to install and configure Docker in Windows, macOS, or Linux: **https://docs.docker.com/install/**.

In the next sections, we will explain how Docker's components work and how to integrate them with our license microservice. In case you didn't follow the example in chapters 1 and 3, you can apply what we're about to explain to any Java Maven project you have.

4.3 Dockerfiles

A *Dockerfile* is a simple text file that contains a list of instructions and commands that the Docker client calls to create and prepare an image. This file automates the image creation process for you. The commands used in the Dockerfile are similar to Linux commands, which makes the Dockerfile easier to understand.

The following code snippet presents a brief example of how a Dockerfile looks. In section 4.5.1, we will show you how to create custom Dockerfiles for your own microservices. Figure 4.4 shows how the Docker image creation workflow should look.

```
FROM openjdk:11-slim
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```



Figure 4.4 Once the Dockerfile is created, you run the `docker build` command to build a Docker image. Then, once the Docker image is ready, you use the `run` command to create the containers.

Table 4.1 shows the most common Dockerfile commands that we'll use in our Dockerfiles. See also listing 4.1 for an example Dockerfile.

Table 4.1 Dockerfile commands

`FROM`	Defines a base image to start the build process. In other words, the `FROM` command specifies the Docker image that you'll use in your Docker run time.
LABEL	Adds metadata to an image. This is a key-value pair.
`ARG`	Defines variables that the user can pass to the builder using the `docker build` command.
COPY	Copies new files, directories, or remote file URLs from a source and adds them to the filesystem of the image we are creating at the specified destination path (for example, COPY ${JAR_FILE} app.jar).
`VOLUME`	Creates a mount point in our container. When we create a new container using the same image, we will create a new volume that will be isolated from the previous one.
RUN	Takes the command and its arguments to run a container from the image. We often use this for installing software packages.
`CMD`	Provides arguments to the `ENTRYPOINT` . This command is similar to the `docker run` command, but this command gets executed only after a container is instantiated.
ADD	Copies and adds the files from a source to a destination within the container.
`ENTRYPOINT`	Configures a container that will run as an executable.
ENV	Sets the environment variables.

4.4 Docker Compose

Docker Compose simplifies the use of Docker by allowing us to create scripts that facilitate the design and the construction of services. With Docker Compose, you can run multiple containers as a single service or you can create different containers simultaneously. To use Docker Compose, follow these steps:

1. Install Docker Compose if you don't have it already installed. You can access this tool at https://docs.docker.com/compose/install/.
2. Create a YAML file to configure your application services. You should name this file docker-compose.yml.
3. Check the validity of the file using the following command: `docker-compose config`.
4. Start the services using the following command: `docker-compose up`.

A docker-compose.yml file should look like that shown in the following listing. Later, in this chapter, we will explain how to create our docker-compose.yml file.

Listing 4.1 A sample docker-compose.yml file

```
version: <docker-compose-version>
services:
  database:
    image: <database-docker-image-name>
    ports:
      - "<databasePort>:<databasePort>"
    environment:
      POSTGRES_USER: <databaseUser>
      POSTGRES_PASSWORD: <databasePassword>
      POSTGRES_DB:<databaseName>

  <service-name>:
    image: <service-docker-image-name>
    ports:
      - "<applicationPort>:<applicationPort>"
    environment:
      PROFILE: <profile-name>
      DATABASESERVER_PORT: "<databasePort>"
    container_name: <container_name>
      networks:
      backend:
      aliases:
        - "alias"

networks:
  backend:
    driver: bridge
```

Table 4.2 shows the instructions that we will use in the docker-compose.yml file. Then table 4.3 lists the `docker-compose` commands that we will use throughout the book.

Table 4.2 Docker Compose instructions

`version`	Specifies the version of the Docker Compose tool.
service	Specifies which services to deploy. The service name becomes the DNS entry for the Docker instance when started and is how other services access it.
`image`	Specifies the tool to run a container using the specified image.
port	Defines the port number on the started Docker container that will be exposed to the outside world. Maps the internal and the external port.
`environment`	Passes the environment variables to the starting Docker image.
network	Specifies a custom network, allowing us to create complex topologies. The default type is `bridge`, so if we don't specify another type of network (`host`, `overlay`, `macvlan`, or `none`), we will create a bridge. The bridge network allows us to maintain the container's connection to the same network. Note that the bridge network only applies to the containers running on the same Docker daemon host.
`alias`	Specifies an alternative hostname for the service on the network.

Table 4.3 Docker Compose commands

`docker-compose up -d`	Builds the images for your application and starts the services you define. This command downloads all the necessary images and then deploys these and starts the container. The `-d` parameter indicates to run Docker in the background.
`docker-compose logs`	Lets you view all the information about your latest deployment.
`docker-compose logs <service_id>`	Lets you view the logs for a specific service. To view the licensing service deployment, for example, use this command: `docker-compose logs licenseService`.
`docker-compose ps`	Outputs the list of all the containers you have deployed in your system.
`docker-compose stop`	Stops your services once you have finished with them. This also stops the containers.
`docker-compose down`	Shuts down everything and removes all containers.

4.5 Integrating Docker with our microservices

Now that we understand the main components of Docker, let's integrate Docker with our licensing microservice to create a more portable, scalable, and manageable microservice. To achieve this integration, let's start by adding the Docker Maven plugin to our licensing service created in the previous chapter. In case you didn't follow the code listings there, you can download the code created in chapter 3 from the following link:

https://github.com/ihuaylupo/manning-smia/tree/master/chapter3

4.5.1 Building the Docker Image

To begin, we'll build a Docker image by adding the Docker Maven plugin to the pom.xml of our licensing service. This plugin will allow us to manage the Docker images and containers from our Maven pom.xml file. The following listing shows how your pom file should look.

Listing 4.2 Adding `dockerfile-maven-plugin` to pom.xml

```
<build>                                          ❶
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <!-- This plugin is used to create a Docker image and publish
       it to Docker hub-->
    <plugin>                                     ❷
      <groupId>com.spotify</groupId>
      <artifactId>dockerfile-maven-plugin</artifactId>
      <version>1.4.13</version>
      <configuration>
        <repository>${docker.image.prefix}/
        ${project.artifactId}</repository>       ❸
        <tag>${project.version}</tag>            ❹
        <buildArgs>
          <JAR_FILE>target/${project.build.finalName}
            .jar</JAR_FILE>                      ❺
        </buildArgs>
      </configuration>
      <executions>
        <execution>
          <id>default</id>
          <phase>install</phase>
          <goals>
            <goal>build</goal>
            <goal>push</goal>
```

```
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

❶ Build section of the pom.xml

❷ Starts the Dockerfile Maven plugin

❸ Sets the remote repository name. Here we use a predefined variable called docker.image.prefix and the project.artifactId.

❹ Sets the repository tag with the project version

❺ Sets the JAR file location using <buildArgs>. This value is used in the Dockerfile.

Now that we have the plugin in our pom.xml file, we can continue the process by creating the variable `docker.image.prefix` mentioned in the previous code listing ❸. This variable will indicate what prefix to use for our image. The following listing shows how to add the variable into the pom.xml.

Listing 4.3 Adding the `docker.image.prefix` variable

```
<properties>
  <java.version>11</java.version>
  <docker.image.prefix>ostock</docker.image.prefix>     ❶
</properties>
```

❶ Sets the value for the docker.image.prefix variable

There are several ways to define the value for the `docker.image.prefix` variable. Listing 4.3 shows but one option. Another way is to directly send the value using the `-d` option in the Maven JVM arguments. Note that if you don't create this variable in the `<properties>` section of the pom.xml, when you execute the command to package and create the Docker image, the following error is thrown:

```
Failed to execute goal com.spotify:dockerfile-maven-plugin:1.4.0:build (default-cli) on project licensing-service: Execution default-cli of goal com.spotif
```

Now that we've imported the plugin into the pom.xml, let's continue by adding the Dockerfile to our project. In the next sections, we will show you how to create a basic Dockerfile and a multistage build Dockerfile. Note that you can use both Dockerfiles because both files allow you to execute your microservice. The main difference between them is that with the basic Dockerfile, you'll copy the entire JAR file of your Spring Boot microservice, and with the multistage build, you'll copy only what's essential to the application. In this book, we chose to use the multistage build to optimize the Docker image that we will create, but feel free to use the option that best suits your needs.

BASIC DOCKERFILE

In this Dockerfile, we will copy the Spring Boot JAR file into the Docker image and then execute the application JAR. The following listing shows how to achieve this with a few simple steps.

Listing 4.4 A basic Dockerfile

```
#Start with a base image containing Java runtime
FROM openjdk:11-slim                    ❶

# Add Maintainer Info
LABEL maintainer="Illary Huaylupo <illaryhs@gmail.com>"

# The application's jar file
ARG JAR_FILE                            ❷

# Add the application's jar to the container
COPY ${JAR_FILE} app.jar                ❸
```

```
#execute the application
ENTRYPOINT ["java","-jar","/app.jar"]                    ❹
```

❶ Specifies the Docker image to use in our Docker run time (in this case, openjdk:11-slim)

❷ Defines the JAR_FILE variable set by dockerfile-maven-plugin

❸ Copies the JAR file to the filesystem of the image named app.jar

❹ Targets the licensing service application in the image when the container is created

MULTISTAGE BUILD DOCKERFILE

In the Dockerfile for this section, we use a multistage build. Why multistage? This will allow us to discard anything that isn't essential to the execution of the application. For example, with Spring Boot, instead of copying the entire target directory to the Docker image, we only need to copy what's necessary to run the Spring Boot application. This practice will optimize the Docker image we create. The following listing shows how your Dockerfile should look.

Listing 4.5 A Dockerfile with a multistage build

```
#stage 1
#Start with a base image containing Java runtime
FROM openjdk:11-slim as build

# Add Maintainer Info
LABEL maintainer="Illary Huaylupo <illaryhs@gmail.com>"

# The application's jar file
ARG JAR_FILE

# Add the application's jar to the container
COPY ${JAR_FILE} app.jar

#unpackage jar file
RUN mkdir -p target/dependency &&
   (cd target/dependency; jar -xf /app.jar)                    ❶

#stage 2                                                       ❷
#Same Java runtime
FROM openjdk:11-slim

#Add volume pointing to /tmp
VOLUME /tmp

#Copy unpackaged application to new container                  ❸
ARG DEPENDENCY=/target/dependency
COPY --from=build ${DEPENDENCY}/BOOT-INF/lib /app/lib
COPY --from=build ${DEPENDENCY}/META-INF /app/META-INF
COPY --from=build ${DEPENDENCY}/BOOT-INF/classes /app

#execute the application                                       ❹
ENTRYPOINT ["java","-cp","app:app/lib/*","com.optimagrowth.license.
        LicenseServiceApplication"]
```

❶ Unpacks the app.jar copied previously into the filesystem of the build image

❷ This new image contains the different layers of a Spring Boot app instead of the complete JAR file.

❸ Copies the different layers from the first image named build

❹ Targets the licensing service in the image when the container is created

We won't go through the entire multistage Docker file in detail, but we'll note a few key areas. In stage 1, using the `FROM` command, the Dockerfile creates an image called `build` from the `openJDK` image that is optimized for Java applications. This image is in charge of creating and unpacking the JAR application file.

NOTE The image we use already has the Java 11 JDK installed on it.

Next, the Dockerfile obtains the value for the `JAR_FILE` variable that we set in the `<configuration> <buildArgs>` section of the pom.xml. Then, we copy the JAR file into the image filesystem as app.jar and unpack it to expose the different layers that a Spring Boot application contains. Once the different layers are exposed, the Dockerfile creates another image that will only contain the layers instead of the complete application JAR. After that, in stage 2, the Dockerfile copies the different layers into the new image.

NOTE If we don't change the project's dependencies, the BOOT-INF/lib folder doesn't change. This folder contains all of the internal and external dependencies needed to run the application.

Finally, the `ENTRYPOINT` command allows us to target the licensing service application in the new image when the container is created. To understand more about the multistage build process, you can take a look inside the Spring Boot application fat JAR by executing the following command on the target folder of your microservice:

```
jar tf jar-file
```

For example, for the licensing service, the command should look like the following:

```
jar tf licensing-service-0.0.1-SNAPSHOT.jar
```

In case you don't have the JAR file in the target folder, execute the following Maven command on the root of the pom.xml file of your project:

```
mvn clean package
```

Now that we have our Maven environment set up, let's build our Docker image. To achieve this, you will need to execute the following command:

```
mvn package dockerfile:build
```

NOTE Verify that you have at least version 18.06.0 or greater of the Docker Engine on your local machine in order to guarantee that all the Docker code examples will run successfully. To find your Docker version, execute the `docker version` command.

Once the Docker image is built, you should see something like that shown in figure 4.5.

```
--- dockerfile-maven-plugin:1.4.13:build (default-cli) @ licensing-service ---
dockerfile: null
contextDirectory: /Users/illary.huaylupo/Documents/Personal/Manning/code/manning-smia/chapter4/licensing-servic
Building Docker context /Users/illary.huaylupo/Documents/Personal/Manning/code/manning-smia/chapter4/licensing-
Path(dockerfile): null
Path(contextDirectory): /Users/illary.huaylupo/Documents/Personal/Manning/code/manning-smia/chapter4/licensing-

Image will be built as ostock/licensing-service:0.0.1-SNAPSHOT

Step 1/12 : FROM openjdk:11-slim as build

Pulling from library/openjdk
Digest: sha256:225e03d0955b1cd6da39003db94f0e655b112b76bd65a29e06e8dd98e9030bf5
Status: Image is up to date for openjdk:11-slim
 ---> 6085fd745c24
Step 2/12 : LABEL maintainer="Illary Huaylupo <illaryhs@gmail.com>"

 ---> Running in 371380f9e53e
Removing intermediate container 371380f9e53e
 ---> 57a44e0e985b
Step 3/12 : ARG JAR_FILE

 ---> Running in 4655b1862836
Removing intermediate container 4655b1862836
 ---> 71e6835e40fc
Step 4/12 : COPY ${JAR_FILE} app.jar

 ---> dee44fe0de6c
Step 5/12 : RUN mkdir -p target/dependency && (cd target/dependency; jar -xf /app.jar)

 ---> Running in 9c23dabae835
Removing intermediate container 9c23dabae835
 ---> f0aa2ff9fedc
Step 6/12 : FROM openjdk:11-slim

Pulling from library/openjdk
Digest: sha256:225e03d0955b1cd6da39003db94f0e655b112b76bd65a29e06e8dd98e9030bf5
Status: Image is up to date for openjdk:11-slim
 ---> 6085fd745c24
Step 7/12 : VOLUME /tmp

 ---> Using cache
 ---> bf23ae387bbd
Step 8/12 : ARG DEPENDENCY=/target/dependency

 ---> Running in 9b6a06b1b495
Removing intermediate container 9b6a06b1b495
 ---> 5dab66558903
Step 9/12 : COPY --from=build ${DEPENDENCY}/BOOT-INF/lib /app/lib

 ---> bb8ecadb4040
Step 10/12 : COPY --from=build ${DEPENDENCY}/META-INF /app/META-INF

 ---> 06ddb7fc4d1e
Step 11/12 : COPY --from=build ${DEPENDENCY}/BOOT-INF/classes /app

 ---> 5ae10ed09222
Step 12/12 : ENTRYPOINT ["java","-cp","app:app/lib/*","com.optimagrowth.license.LicenseServiceApplication"]

 ---> Running in 464f8b4b4a45
Removing intermediate container 464f8b4b4a45
 ---> 52cef232a505
Successfully built 52cef232a505
Successfully tagged ostock/licensing-service:0.0.1-SNAPSHOT

Detected build of image with id 52cef232a505
Building jar: /Users/illary.huaylupo/Documents/Personal/Manning/code/manning-smia/chapter4/licensing-service/
target/licensing-service-0.0.1-SNAPSHOT-docker-info.jar
Successfully built ostock/licensing-service:0.0.1-SNAPSHOT
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  21.505 s
Finished at: 2019-12-30T12:18:45-06:00
------------------------------------------------------------------------
```


Figure 4.5 Docker image built with the Maven plugin by executing the `mvn package dockerfile:build` command

Now that we have the Docker image, we can see it in the list of the Docker images on our system. To list all of the Docker images, we need to execute the `docker images` command. If everything runs correctly, you should see something like this:

```
REPOSITORY                TAG       IMAGE ID      CREATED         SIZE
ostock/licensing-service  0.0.1-SNAPSHOT  231fc4a87903  1 minute ago  149MB
```

Once we have the Docker image, we can run it by using the following command:

```
docker run ostock/licensing-service:0.0.1-SNAPSHOT
```

You can also use the `-d` option in the `docker run` command to run the container in the background. For example, like this:

```
docker run -d ostock/licensing-service:0.0.1-SNAPSHOT
```

This `docker run` command starts the container. In order to see all the running containers in your system, you can execute the `docker ps` command. That command opens all the running containers with their corresponding ID, image, command, creation date, status, port, and name. If you need to stop the container, you can execute the following command with the corresponding container ID:

```
docker stop <container_id>
```

4.5.2 Creating Docker images with Spring Boot

In this section, we will give you a brief overview of how to create Docker images using the new features released in Spring Boot v2.3. Note that in order to use these new features, you must

- Have Docker and Docker Compose installed.
- Have a Spring Boot application with a version equal to or greater than 2.3 in your microservice.

These new features help improve the Buildpack support and layered JARs. To create a Docker image using the new features, add the following. Make sure your pom.xml file contains the `spring-boot-starter-parent` version 2.3 or greater.

```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.4.0</version>
<relativePath/> <!-- lookup parent from repository -->
</parent>
```

BUILDPACKS

Buildpacks are tools that provide application and framework dependencies, transforming our source code into a runnable application image. In other words, Buildpacks detect and obtain everything the application needs to run.

Spring Boot 2.3.0 introduces support for building Docker images using Cloud Native Buildpacks. This support was added to the Maven and Gradle plugins using the `spring-boot:build-image` goal for Maven and the `bootBuildImage` task for Gradle. For more information, see the following links:

- Spring Boot Maven Plugin at **https://docs.spring.io/spring-boot/docs/2.3.0.M1/maven-plugin/html/**
- Spring Boot Gradle Plugin at **https://docs.spring.io/spring-boot/docs/2.3.0.M1/gradle-plugin/reference/html/**

In this book, we will only explain how to use the Maven scenario. To build the image using this new feature, execute the following command from the root directory of your Spring Boot microservice:

```
./mvnw spring-boot:build-image
```

Once the command executes, you should be able to see output similar to that shown in the next code snippet:

```
[INFO]    [creator]    Setting default process type 'web'
[INFO]    [creator]    *** Images (045116f040d2):
[INFO]    [creator]    docker.io/library/licensing-service:0.0.1-SNAPSHOT
[INFO]
[INFO] Successfully built image 'docker.io/library/
➥ licensing-service:0.0.1-SNAPSHOT'
```

If you want to customize the name of the image created, you can add the following plugin to your pom.xml file and then define the name under the configuration section:

```
<plugin>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-maven-plugin</artifactId>
   <configuration>
   <image>
      <name>${docker.image.prefix}/${project.artifactId}:latest</name>
   </image>
   </configuration>
</plugin>
```

Once the image is successfully built, you can execute the following command to start the container via Docker:

```
docker run -it -p8080:8080 docker.io/library/
➥ licensing-service:0.0.1-SNAPSHOT
```

LAYERED JARS

Spring Boot introduced a new JAR layout called layered JARs. In this format, the /lib and /classes folders are split up and categorized into layers. This layering was created to separate code based on how likely it is to change between the builds, leaving the necessary information for the build. This is another excellent option to use in case you don't want to use the Buildpacks. To extract the layers of our microservice, let's execute the following steps:

1. Add the layer configuration into the pom.xml file
2. Package the application
3. Execute the `jarmode` system property with the `layertools` JAR mode
4. Create the Dockerfile
5. Build and run the image

The first step adds the layer configuration to the Spring Boot Maven plugin in pom.xml, for example:

```
<plugin>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-maven-plugin</artifactId>
   <configuration>
        <layers>
        <enabled>true</enabled>
      </layers>
   </configuration>
</plugin>yp
```

Once we have the configuration set up in our pom.xml file, we can execute the following command to rebuild our Spring Boot JAR:

```
mvn clean package
```

Once the JAR file is created, we can execute the following command in the root directory of our application to display the layers and the order in which these should be added to our Dockerfile:

```
java -Djarmode=layertools -jar target/
➥ licensing-service-0.0.1-SNAPSHOT.jar list
```

And, once executed, you should see output similar to the following code snippet:

```
dependencies
spring-boot-loader
snapshot-dependencies
application
```

Now that we have the layer information, let's continue with the fourth step, which is creating our Dockerfile. The following listing shows you how.

Listing 4.6 Creating a Dockerfile file with layered JARS

```
FROM openjdk:11-slim as build
WORKDIR application
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} application.jar
RUN java -Djarmode=layertools -jar application.jar extract

FROM openjdk:11-slim
WORKDIR application
COPY --from=build application/dependencies/ ./          (1)
COPY --from=build application/spring-boot-loader/ ./
```

```
COPY --from=build application/snapshot-dependencies/ ./
COPY --from=build application/application/ ./                    ❶
ENTRYPOINT ["java", "org.springframework.boot.loader
        .JarLauncher"]                                           ❷
```

❶ Copies each layer displayed as a result of the jarmode command

❷ Uses org.springframework.boot.loader.JarLauncher to execute the application

Finally, we can execute the `build` and `run` Docker commands in the root directory of our microservice:

```
docker build . --tag licensing-service
docker run -it -p8080:8080 licensing-service:latest
```

4.5.3 Launching the services with Docker Compose

Docker Compose is installed as part of the Docker installation process. It's a service orchestration tool that allows you to define services as a group and then to launch these together as a single unit. Docker Compose includes capabilities for also defining environment variables for each service.

Docker Compose uses a YAML file for defining the services that are going to be launched. For example, each chapter in this book has a file called <<*chapter*>>/docker-compose.yml. This file contains the service definitions used to launch the services for the chapter.

Let's create our first docker-compose.yml file. The following listing shows what your docker-compose.yml should look like.

Listing 4.7 The docker-compose.yml file

```
version: '3.7'

services:
  licensingservice:                                  ❶

    image: ostock/licensing-service:0.0.1-SNAPSHOT   ❷

    ports:
      - "8080:8080"                                  ❸

    environment:
      - "SPRING_PROFILES_ACTIVE=dev"                 ❹

    networks:
      backend:                                       ❺
        aliases:
          - "licenseservice"                         ❻

networks:
  backend:                                           ❼
    driver: bridge
```

❶ Applies a label to each service launched. This becomes the DNS entry for the Docker instance when it starts, which is how other services access it.

❷ Docker Compose first tries to find the target image to be started in the local Docker repository. If it can't find it, it checks the central Docker Hub (**http://hub.docker.com**).

❸ Defines the port numbers on the started Docker container, which are exposed to the outside world

❹ Passes along environment variables to the starting Docker image. In this case, sets the SPRING_PROFILES_ACTIVE environment variable on the starting Docker image.

❺ Names the network where the service belongs

❻ Specifies the alternative hostname for the service on the network

❼ Creates a custom network named backend with the default type bridge

Now that we have a docker-compose.yml, we can start our services by executing the `docker-compose up` command in the directory where the docker-compose.yml file is located. Once it executes, you should see a result similar to that shown figure 4.6.

NOTE If you are not yet familiar with the `SPRING_PROFILES_ACTIVE` variable, don't worry about it. We'll cover this in the next chapter, where we'll manage different profiles in our microservice.

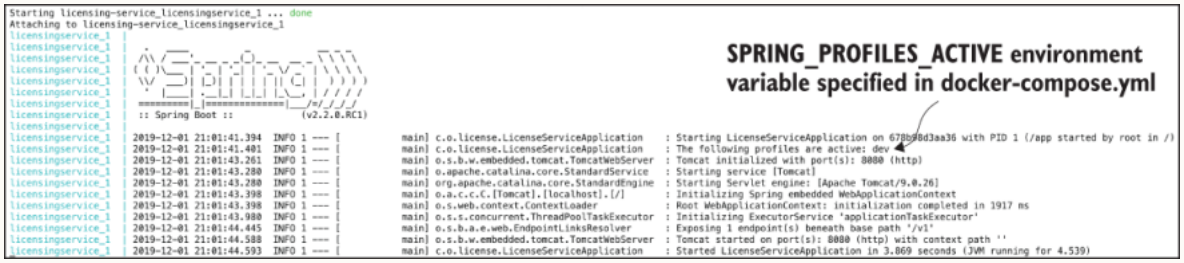


Figure 4.6 The Docker Compose console log, showing that the licensing service is up and running with the `SPRING_PROFILES_ACTIVE` variable specified in docker-compose.yml

Once you start the container, you can execute the `docker ps` command to see all the containers that are running.

NOTE All the Docker containers used in this book are *ephemeral*—they won't retain their state after they're started and stopped. Keep this in mind if you start playing with code and you see your data disappear after your restart your containers. If you are interested in how to save the container state, take a look at the `docker commit` command.

Now that we know what containers are and how to integrate Docker with our microservices, let's continue with the next chapter. In that chapter, we'll create our Spring Cloud configuration server.

Summary

- Containers allow us to execute successfully the software we are developing in any environment—from the developer's machine to a physical or virtual enterprise server.
- A virtual machine (VM) allows us to emulate the operation of a computer within another computer. This is based on a hypervisor that mimics the complete physical machine, allocating the desired amount of system memory, processor cores, disk storage, and other resources such as networks, PCI add-ons, and so forth.
- Containers are another operating system (OS) virtualization method that allows you to run an application with its dependent elements in an isolated and independent environment.
- The use of containers reduces general costs by creating lightweight VMs that speed up the run/execution process, therefore reducing the costs of each project.
- Docker is a popular open source container engine based on Linux containers. It was created by Solomon Hykes, founder and CEO of dotCloud in 2013.
- Docker is composed of the following components: Docker Engines, clients, Registries, images, containers, volumes, and networks.
- A Dockerfile is a simple text file that contains a list of instructions and commands that the Docker client calls to create and prepare an image. This file automates the image creation process. The commands used in the Dockerfile are similar to Linux commands, which makes the Dockerfile commands easier to understand.
- Docker Compose is a service orchestration tool that allows you to define services as a group and then launch them together as a single unit.
- Docker Compose is installed as part of the Docker installation process.
- The Dockerfile Maven plugin integrates Maven with Docker.

Previous chapter
< 3 Building microservices with Spring Boot

Next chapter
5 Controlling your configuration with the Spring Cloud Configuration Server >